COMP9315 23T1 DBMS Implementation

Exercises 04

Implementing Sorting and Projection

[Show with no answers] [Show with all answers]

1. You have an unsorted heap file containing 4500 records and a select query is asked that requires the file to be sorted. The DBMS uses an external merge-sort that makes efficient use of the available buffer space.

   Assume that: records are 48-bytes long (including a 4-byte sort key); the page size is 512-bytes; each page has 12 bytes of control information in it; 4 buffer pages are available.

   a. How many sorted subfiles will there be after the initial pass of the sort algorithm? How long will each subfile be?

      [show answer]

   b. How many passes (including the initial pass considered above) will be required to sort this file?

      [show answer]

   c. What will be the total I/O cost for sorting this file?

      [show answer]

   d. What is the largest file, in terms of the number of records, that you can sort with just 4 buffer pages in 2 passes? How would your answer change if you had 257 buffer pages?

      [show answer]

2. For each of these scenarios:

   i. a file with 10,000 pages and 3 available buffer pages.
   ii. a file with 20,000 pages and 5 available buffer pages.
   iii. a file with 2,000,000 pages and 17 available buffer pages.

   answer the following questions assuming that external merge-sort is used to sort each of the files:

   a. How many runs will you produce on the first pass?

      [show answer]

   b. How many passes will it take to sort the file completely?

[show answer]

c. What is the total I/O cost for sorting the file? (measured in #pages read/written)

[show answer]

3. Consider processing the following SQL projection query:

```
select distinct course from Students;
```

where there are only 10 distinct values (0..9) for `Student.course`, and student enrolments are distributed over the courses as follows:

cid	course	#students	cid	course	#students
0	BSc	5,000	1	BA	4,000
2	BE	5,000	3	BCom	3,000
4	BAppSc	2,000	5	LLB	1,000
6	MA	1,000	7	MSc	1,000
8	MEng	2,000	9	PhD	1,000

Show the flow of records among the pages (buffers and files) when a hash-based implementation of projection is used to eliminate duplicates in this relation.

Assume that:

- each page (data file page or in-memory buffer) can hold 1000 records
- the partitioning phase uses 1 input buffer, 3 output buffers and the hash function `(x mod 3)`
- the duplicate elimination phase uses 1 input buffer, 4 output buffers and the hash function `(x mod 4)`

[show answer]

4. Consider processing the following SQL projection query:

```
select distinct title,name from Staff;
```

You are given the following information:

- the relation `Staff` has attributes `id`, `name`, `title`, `dept`, `address`
- except for `id`, all are string fields of the same length
- the `id` attribute is the primary key
- the relation contains 10,000 pages
- there are 10 records per page

- there are 10 memory buffer pages available for sorting

Consider an optimised version of the sorting-based projection algorithm: The initial sorting pass reads the input and creates sorted runs of tuples containing only the attributes `name` and `title`. Subsequent merging passes eliminate duplicates while merging the initial runs to obtain a single sorted result (as opposed to doing a separate pass to eliminate duplicates from a sorted result).

a. How many sorted runs are produced on the first pass? What is the average length of these runs?

[show answer]

b. How many additional passes will be required to compute the final result of the projection query? What is the I/O cost of these additional passes?

[show answer]

c. Suppose that a clustered B+ tree index on `title` is available. Is this index likely to offer a cheaper alternative to sorting? Would your answer change if the index were unclustered?

[show answer]

d. Suppose that a clustered B+ tree index on `name` is available. Is this index likely to offer a cheaper alternative to sorting? Would your answer change if the index were unclustered?

[show answer]

e. Suppose that a clustered B+ tree index on `(name,title)` is available. Is this index likely to offer a cheaper alternative to sorting? Would your answer change if the index were unclustered?

[show answer]